

Guía Pedagógica: Día 8 - Buscador Inteligente con Signals

Hoy implementaremos una de las características que más disfrutaban los usuarios en aplicaciones modernas: un buscador en tiempo real.

En lugar de tener que escribir el nombre de la película y presionar un botón de "Buscar", nuestra aplicación reaccionará **instantáneamente** mientras el usuario escribe, sugiriendo resultados en un menú desplegable directamente desde el encabezado (Header). Para lograr esta magia reactiva, utilizaremos la característica estrella que se empezó a utilizar a partir de Angular 16+: los **Signals** y los **Effects**.

Contexto Pedagógico: Signals vs Events

Históricamente, para capturar lo que el usuario escribe, usábamos eventos ((keyup)) y llamábamos funciones complejas.

Con **Signals**, pensamos en el "estado" de nuestra aplicación.

1. `searchQuery`: Es una Signal que almacena *exactamente* lo que está escrito en la barra de búsqueda en todo momento.
2. `effect`: Es un vigía de Angular. Le decimos: *"Oye, observa searchQuery. Si cambia, ejecuta este código"*.

Esta forma de pensar hace que el código sea mucho más limpio, reactivo y fácil de mantener.

Paso 1: El Servicio - Buscar en TMDb

Nuestra API necesita saber qué queremos buscar. Abriremos nuestro "mensajero" para añadir esta nueva capacidad.

Abre **src/app/core/services/movie.service.ts** y añade el siguiente método:

```

/**
 * Busca películas por término de búsqueda.
 * @param query Texto a buscar
 */
searchMovies(query: string) {
  return this.http.get<MovieResponse>
(`${this.apiUrl}/search/movie`, {
    params: { query } // Angular convierte esto en ?query=Batman
    automáticamente
  });
}

```

Paso 2: La Lógica del Buscador (El Cerebro)

Toda la magia ocurrirá en nuestro componente de navegación principal.

Abre `src/app/shared/components/layout/header/header.ts`. Vamos a reescribirlo para que maneje la búsqueda usando Signals.

```

import { Component, effect, inject, signal } from '@angular/core';
import { RouterLink, RouterLinkActive } from '@angular/router';
import { MovieService } from '../../../core/services/movie.service';
import { Movie } from '../../../core/models/movie.model';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-header',
  standalone: true,
  imports: [RouterLink, RouterLinkActive, CommonModule],
  templateUrl: './header.html',
  styleUrls: ['./header.css']
})
export class Header {
  private movieService = inject(MovieService);

  // 1. Declaramos nuestras Signals
  searchQuery = signal(''); // lo que el usuario escribe
  searchResults = signal<Movie[]>([]); // los resultados que llegan de la API
  isSearching = signal(false); // Para mostrar un spinner de carga

  constructor() {
    // 2. El Effect: Reacciona automáticamente cuando searchQuery cambia
    effect((onCleanup) => {
      const query = this.searchQuery();

      // Si el texto es muy corto, no buscamos (ahorramos peticiones)
      if (query.length < 3) {
        this.searchResults.set([]);
        this.isSearching.set(false);
        return;
      }

      this.isSearching.set(true);

      // 3. El "Debounce": Retrasamos la búsqueda 300ms
      // ¿Por qué? Si el usuario escribe "Batman" muy rápido (6 letras),
      const timeoutId = setTimeout(() => {
        this.movieService.searchMovies(query).subscribe({
          next: (response) => {
            // Tomamos solo los primeros 5 resultados para no llenar la pantalla
            this.searchResults.set(response.results.slice(0, 5));
            this.isSearching.set(false);
          },
          error: () => this.isSearching.set(false)
        });
      }, 300);

      // 4. Limpieza: Si el usuario escribe OTRA letra antes de los 300ms,
      // cancelamos el temporizador anterior.
      onCleanup(() => clearTimeout(timeoutId));
    });

    // Método que conectaremos al HTML para actualizar la signal
    onSearchInput(event: Event) {
      const input = event.target as HTMLInputElement;
      this.searchQuery.set(input.value);
    }

    // Método para cerrar el buscador al hacer clic en un resultado
    closeSearch() {
      this.searchQuery.set('');
      this.searchResults.set([]);
    }
  }
}

```

Paso 3: La Vista - Input y Dropdown (HTML)

Abre **src/app/shared/components/layout/header/header.html**. Vamos a cambiar el viejo enlace de "Buscar" por un input real y un contenedor de resultados.

Reemplaza tu HTML actual con este:

```
<nav class="navbar">
  <div class="navbar-content">
    <div class="logo">
      <span class="logo-text">Movie<span>Nexus</span></span>
    </div>

    <ul class="nav-links">
      <li><a routerLink="/" routerLinkActive="active" [routerLinkActiveOptions]="{exact:
true}">Inicio</a></li>
      <!-- El enlace de 'Buscar' fue eliminado, ahora está integrado en el Header -->
      <li><a routerLink="/favorites" routerLinkActive="active">Favoritos</a></li>
    </ul>

    <!-- NUEVO: Contenedor del Buscador -->
    <div class="search-container">

      <!-- El Input -->
      <input
        type="text"
        class="search-input"
        placeholder="Buscar películas..."
        [value]="searchQuery()"
        (input)="onSearchInput($event)"
      >

      <!-- Indicador de carga (Spinner) -->
      @if (isSearching()) {
        <div class="search-spinner"></div>
      }

      <!-- Dropdown de resultados -->
      @if (searchResults().length > 0) {
        <div class="search-results-dropdown">
          @for (movie of searchResults(); track movie.id) {
            <!-- Cada resultado es un enlace a los detalles -->
            <a
              [routerLink]="['/movie', movie.id]"
              class="search-result-item"
              (click)="closeSearch()"
            >
              <img
                [src]="movie.poster_path ? 'https://image.tmdb.org/t/p/w92' + movie.poster_path :
'assets/no-poster.png'"
                [alt]="movie.title"
                class="result-poster"
              >
              <div class="result-info">
                <span class="result-title">{{ movie.title }}</span>
                <span class="result-year">{{ movie.release_date | date:'yyyy' }}</span>
              </div>
            </a>
          }
        </div>
      }
    </div>
  </div>
</nav>
```

4. Los Estilos del Buscador (CSS)

Abre **src/app/shared/components/layout/header/header.css** y añade al final los siguientes estilos para que el buscador y el dropdown luzcan increíbles con un efecto tipo Glassmorphism.

```
/* Buscador */
.search-container {
  position: relative;
  margin-left: 2rem;
}

.search-input {
  background: rgba(255, 255, 255, 0.1);
  border: 1px solid rgba(255, 255, 255, 0.2);
  border-radius: 20px;
  padding: 0.6rem 1rem;
  color: white;
  font-size: 0.95rem;
  width: 250px;
  outline: none;
  transition: all 0.3s ease;
}

.search-input:focus {
  background: rgba(255, 255, 255, 0.15);
  border-color: rgba(255, 255, 255, 0.4);
  box-shadow: 0 0 10px rgba(229, 9, 20, 0.2);
}

.search-spinner {
  position: absolute;
  right: 12px;
  top: 50%;
  transform: translateY(-50%);
  width: 16px;
  height: 16px;
  border: 2px solid rgba(255, 255, 255, 0.2);
  border-top-color: #e50914;
  border-radius: 50%;
  animation: spin 0.8s linear infinite;
}
```

```
@keyframes spin {
  to { transform: translateY(-50%) rotate(360deg); }
}

/* Dropdown de Resultados */
.search-results-dropdown {
  position: absolute;
  top: 100%;
  left: 0;
  right: 0;
  margin-top: 0.5rem;
  background: rgba(20, 20, 20, 0.95);
  backdrop-filter: blur(10px);
  border: 1px solid rgba(255, 255, 255, 0.1);
  border-radius: 12px;
  max-height: 400px;
  overflow-y: auto;
  box-shadow: 0 10px 30px rgba(0, 0, 0, 0.5);
  z-index: 1000;
}

.search-result-item {
  display: flex;
  padding: 0.8rem;
  gap: 1rem;
  border-bottom: 1px solid rgba(255, 255, 255, 0.05);
  transition: background 0.2s;
  text-decoration: none;
  color: white;
}

.search-result-item:hover {
  background: rgba(255, 255, 255, 0.1);
}

.result-poster {
  width: 40px;
  height: 60px;
  object-fit: cover;
  border-radius: 4px;
}

.result-info {
  display: flex;
  flex-direction: column;
  justify-content: center;
}

.result-title {
  font-weight: 600;
  font-size: 0.9rem;
  margin-bottom: 0.2rem;
}

.result-year {
  font-size: 0.8rem;
  color: #aaa;
}

/* Scrollbar para el dropdown */
.search-results-dropdown::-webkit-scrollbar {
  width: 6px;
}

.search-results-dropdown::-webkit-scrollbar-thumb {
  background: rgba(255, 255, 255, 0.2);
  border-radius: 10px;
}
```

Prueba de Aprendizaje (Checklist)

1. **¿Qué es y para qué sirve un effect en Angular Signals?** (Respuesta: Es una función que observa automáticamente una o más Signals. Cuando el valor de esa Signal cambia, el effect vuelve a ejecutar el código en su interior. Lo usamos aquí para reaccionar a los cambios en la barra de búsqueda).
2. **¿Qué es el patrón "Debounce" (el uso de setTimeout y clearTimeout en el effect)?** (Respuesta: Es una técnica para evitar hacer muchas tareas pesadas seguidas. Retrasa la petición a la API unos milisegundos y, si el usuario sigue escribiendo, cancela la petición anterior. Esto ahorra cientos de llamadas innecesarias a la API).
3. **¿Para qué sirve onCleanup dentro del effect?** (Respuesta: Permite ejecutar un código de "limpieza" justo antes de que el effect se vuelva a ejecutar. En nuestro caso, lo usamos para cancelar el temporizador anterior con clearTimeout).



Control de Versiones

Recuerda subir el avance:

```
git add .
git commit -m "feat: implementar buscador inteligente en el navbar usando signals"
git push origin main
```

